



Mechanistic Interpretability

방법론 교육자료

LLM 내부 표상 분석을 위한 9가지 핵심 기법

연구팀 내부 교육용 · 2026

CONTENTS

목차

01 도입 & 이론적 기반

02 Logit Difference

03 Logit Lens & Tuned Lens

04 Cosine Distance

05 Linear Probing & Selectivity

06 Activation Patching & Causal Tracing

07 Attention 분석

08 응용 & 방법론 선택 가이드

MI 개요 / Linear Representation

출력 로짓 분석

예측 궤적 추적

표상 거리 측정

정보량 측정

인과 검증

정보 흐름 추적

Hate Speech 도메인

01

도입 & 이론적 기반

*Mechanistic Interpretability*란 무엇이며 어떤 이론에 기반하는가

Mechanistic Interpretability란

블랙박스인 신경망의 내부를 *reverse engineering*

핵심 질문

어떻게 그런 출력이
나오는가?

성능 측정이 아니라
모델 내부의 어떤 구성요소가
어떤 역할을 하는지를 분석

비교

전통 NLP 평가

정확도/F1만 봄

Saliency map

어떤 입력이 중요한지만

MI (우리 분야)

내부 구조를 직접 분석

4단계 분석 위계

MI 연구는 행동 관찰에서 시작해 인과 검증으로 나아간다

Level 1

행동 수준

차이가 있다

출력 비교

Level 2

위치 특정

어디서 차이가 생기는가

Logit Lens · Probing

Level 3

메커니즘

어떻게 차이가 생기는가

Sentiment direction · Attention

Level 4

인과

그것 때문에 차이가 생긴다

Activation Patching

진정한 mechanistic 주장은 Level 4까지 가야 한다 — 우리 연구는 현재 Level 2

Linear Representation Hypothesis

중요한 개념은 활성화 공간에서 선형 방향으로 인코딩된다

3072차원 hidden state 안에 sentiment 방향 · truth 방향 · 혐오 방향 같은 의미 축이 직선 형태로 존재한다

왜 중요한가

- 비선형이면 분석 매우 어려움
- 선형이면 단순 분류기로 측정 가능
- 벡터 연산으로 조작 가능

검증 방법

$v_sent = \text{mean}(h+) - \text{mean}(h-)$

$score = h_new \cdot v_sent$

이 score 하나만으로 분류 가능?

02

Logit Difference

출력 로짓 분석 – 가장 기본적인 측정 도구

두 토큰 간 점수 차이를 측정

Softmax 통과 전의 원시 점수(logit)를 직접 비교

$$LD = \text{logit}(\text{token_A}) - \text{logit}(\text{token_B})$$

우리 연구에서의 사용

프롬프트: I really enjoyed this. The sentiment of this text is ____

측정: $\text{logit}(\text{negative}) - \text{logit}(\text{positive})$

결과: 양수 → 모델이 negative로 판단 · 음수 → positive로 판단

⚠ 한계: 최종 출력만 본다 → 어느 레이어에서 판단이 형성되는지 모름

왜 Softmax 확률 대신 Logit Difference인가

선형성 · 단조성 · 수치 안정성 세 가지 장점

01

선형성

LD는 lm_head 의 차이이므로 hidden state에 대해 선형. 분해/패칭 분석에 직접 적합.

02

단조성

Softmax는 다른 모든 토큰의 영향을 받지만, LD는 두 토큰만 본다. 다른 토큰 노이즈에 강건.

03

수치 안정성

매우 작거나 큰 확률에서도 의미 있는 차이 측정 가능. log space에서 일정함.

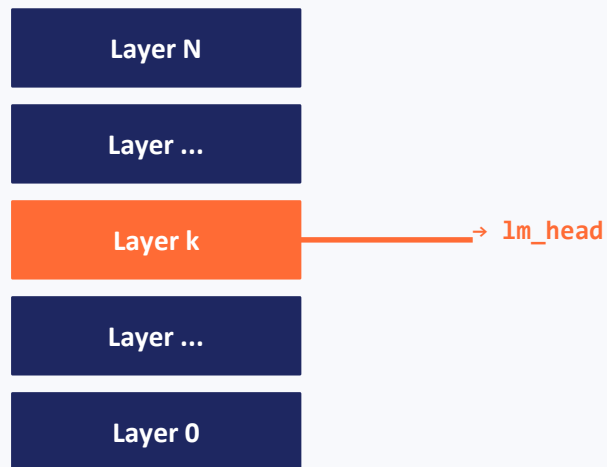
03

Logit Lens

레이어별 예측 궤적을 들여다보는 가장 단순한 도구

각 레이어에서 모델이 떠올리는 답을 들여다본다

중간 hidden state를 최종 lm_head로 디코딩



핵심 아이디어

$\text{LogitLens}(h_k) = \text{softmax}(\text{LN}(h_k) \cdot W_{\text{unembed}})$

얻는 것

- 예측 궤적 (어떻게 변하는가)
- 분기점 (언제 갈라지는가)
- 수렴 시점 (언제 확정되는가)

예측 궤적이 보여주는 패턴

초반엔 빈도 토큰 → 점차 task에 맞는 토큰으로 수렴

Layer 0

"the", "a", "of" ...

의미 없는 고빈도 토큰

Layer 5

"good", "bad", "ok"

대략적 sentiment 방향

Layer 10

"positive", "negative" ...

task에 맞는 토큰 등장

Layer 28

"positive"

최종 답 확정

⚠ 한계: GPT-Neo · BLOOM · OPT 등에선 초기/중반 예측이 무의미함 → Tuned Lens로 보완

Tuned Lens — Logit Lens의 개선판

각 레이어마다 affine 변환을 학습

Logit Lens

$\text{LogitLens}(h_k) = \text{softmax}(\text{LN}(h_k) \cdot W_u)$

- ✓ 단순, 추가 학습 X
- X 일부 모델에서 무의미
- X 편향 강함
- X untied weights에서 약함

Tuned Lens

$\text{TunedLens}(h_k) = \text{softmax}(\text{LN}(A_k \cdot h_k + b_k) \cdot W_u)$

- ✓ 깔끔한 예측 궤적
- ✓ 모델 종류에 robust
- ✓ 편향 감소
- X 사전 학습 가중치 필요

언제 쓰는가: Logit Lens 결과가 노이즈 많을 때 · mechanistic 주장 강화 시

Belrose et al. (2023) — Eliciting Latent Predictions from Transformers with the Tuned Lens

04

Cosine Distance

두 hidden state의 표상 거리를 측정

방향만 보고 크기는 무시한다

두 벡터가 얼마나 다른 방향을 가리키는가

$$\text{cos_sim}(A, B) = (A \cdot B) / (|A| \times |B|) = \cos(\theta)$$

$$\text{cos_dist} = 1 - \text{cos_sim}$$

범위: 0 (완전 동일 방향) ~ 2 (반대 방향)

왜 Euclidean 대신 Cosine?

크기 무시

LLM은 레이어 갈수록 norm 커짐. 방향만 봐야 의미 비교 가능

차원 무관

3072차원 같은 고차원에서도 안정적으로 작동

표준성

word embedding 시대부터 의미적 유사도의 표준

어떤 토큰을 대표로 쓸 것인가

Pooling 방법에 따라 측정하는 게 달라진다

방법	어떻게	장점	단점
Mean Pooling	모든 토큰 평균	안정적, 노이즈 적음	특정 위치 정보 문힘
Last Token	마지막 토큰만	모델이 예측에 쓰는 표상	한 토큰에 의존
Target Token	관심 위치만	관심 정보 보존	위치 식별 필요, 노이즈

우리 연구 결과: Layer 0 피크(~ 0.25) $\rightarrow$ Layer 1-2 수렴(~ 0) $\rightarrow$ Layer 20+ 재증가(~ 0.15)

05

Probing & Selectivity

표상 안에 어떤 정보가 들어 있는지 측정

선형 분류기로 정보량 측정

분류기가 단순할수록 representation 자체를 측정



왜 선형으로 제한하는가

복잡한 분류기는 hidden state에 없는 정보도 만들어낼 수 있음

선형 분류기 정확도 $\approx$ representation 자체의 선형 정보량
복잡한 분류기 정확도 = 정보 + 분류기 학습 능력

⚠ 한계: 분류기가 데이터를 암기할 수 있음 $\rightarrow$ 다음 슬라이드의 Selectivity로 통제 필요

Tenney et al. (2019) · Alain & Bengio (2016)

단순 암기 vs 진짜 정보

Control task와 비교해서 분류기가 실제로 정보를 사용하는지 검증

$$\text{Selectivity} = \text{Task accuracy} - \text{Control accuracy}$$

Selectivity 높음 ○

Task accuracy: 0.97
Control accuracy: 0.50
Selectivity: 0.47

→ 진짜 정보 사용

Selectivity 낮음 ×

Task accuracy: 0.97
Control accuracy: 0.95
Selectivity: 0.02

→ 분류기가 그냥 외운 것

우리 연구: Cell A vs D selectivity ≈ 0.5 (전 레이어 안정) — 진짜 정보가 representation에 있음

Hewitt & Liang (2019) — Designing and Interpreting Probes with Control Tasks

06

Activation Patching

상관관계를 인과관계로 — MI의 가장 강력한 도구

특정 레이어를 바꿔치기하고 출력 변화 측정

이 레이어가 차이를 만든다 → 인과 입증

Clean run (Cell A)

logit_A

Corrupted run (Cell D)

logit_D

Patched run

Cell A 입력, layer k의 activation만 Cell D로 교체

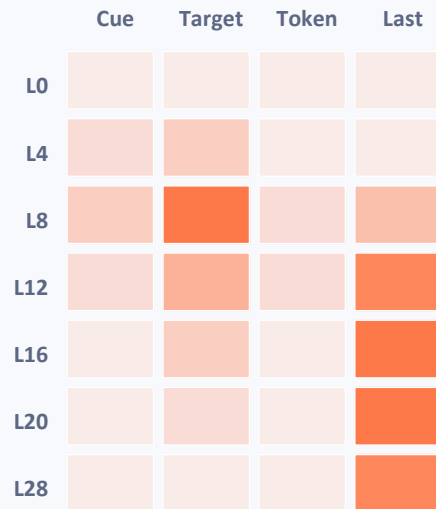
$$\text{effect}(k) = (\text{logit_patched} - \text{logit_clean}) / (\text{logit_corrupted} - \text{logit_clean})$$

0: layer k 영향 없음 · 1: layer k가 차이를 완전히 결정

Meng et al. (2022) · Vig et al. (2020)

2D Heatmap — 레이어 × 위치 동시 특정

Activation Patching *의/systematic 버전*



읽는 법

- 가로: 토큰 위치 (cue · target · last)
- 세로: 레이어 (0~28)
- 색상: patching effect (진할수록 인과적 기여 큼)

Meng et al. (2022) 발견

Subject token의 mid-layer MLP에서 effect 집중. 정보 흐름의 시작점과 경로를 동시에 식별.

07

Attention 분석

정보 흐름과 head 별 역할 추적

Attention 분석 — 4가지 수준

단순 attention weight는 약함. 인과적 분석이 강함

약함

Raw Attention

"attention 높음 ≠ 중요함"

Jain & Wallace (2019) 비판

중간

Attention Rollout

레이어 누적 정보 흐름

Abnar & Zuidema (2020)

중상

Gradient-weighted

예측에 실제 기여한 attention

raw attention의 보완

강함 (인과)

Attention Knockout

head 마스킹 후 출력 변화 측정

Wang et al. (2022) — IOI circuit

우리 다음 단계: Layer 8의 attention knockout으로 핵심 head 식별

08

응용 & 선택 가이드

Hate Speech 도메인과 방법론 선택

LLM은 왜 implicit hate를 놓치는가

선행 연구가 행동 수준에서 확인한 현상

Roy et al. (2023)

Probing LLMs for hate speech detection

Target group 정보를 명시적으로 제공하면 성능이 20-30% 향상

→ 모델은 target 정보를 충분히 활용하지 못한다

Zhang et al. (2024)

Don't Go To Extremes

Toxic word 없으면 implicit hate detection 실패, 있으면 무해 문장도 toxic 분류

→ Cue에 과의존하는 패턴

우리 연구: 행동 수준 한계의 메커니즘을 내부 표상 수준에서 분석

단계별 추천 워크플로우

현상 → 위치 → 메커니즘 → 인과 순서로 진행

1	현상 확인	Logit Difference · 효과 크기 측정	<i>Cohen's d, Wilcoxon</i>
2	위치 좁히기	Logit Lens · Cosine Distance	<i>두 분석이 같은 layer 가리키는지</i>
3	정보량 확인	Linear Probing · Selectivity	<i>단순 암기 가능성 배제</i>
4	메커니즘 가설	Sentiment direction · Target token 분석	<i>Attention rollout</i>
5	인과 검증	Activation Patching · Causal Tracing	<i>Attention Knockout</i>

흔한 실수 5가지

이렇게 하면 결과 해석이 무너진다

- X Probing accuracy만으로 정보량 판단
- X Logit Lens 결과를 모든 모델에서 신뢰
- X Raw attention으로 '중요도' 주장
- X Mean pooling만 사용
- X 상관관계를 인과관계로 해석

→ Selectivity 함께 측정 필수

→ GPT-Neo · BLOOM 등은 안 맞음

→ Jain & Wallace 비판 고려

→ Target token 위치 차이 문힘

→ Patching 없이 '원인' 주장 불가

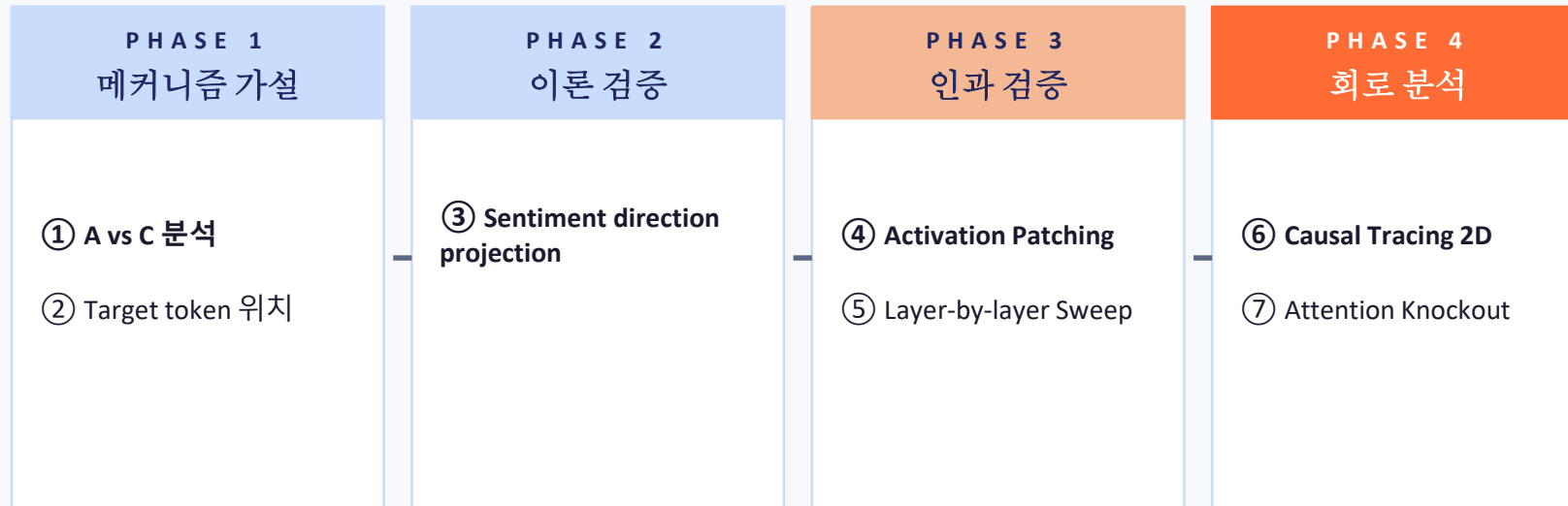
09

앞으로의 실험

8개 실험으로 Level 2 → Level 4 인과 검증까지

전체 실험 로드맵

단기 → 중기 → 장기 순으로 5 단계 진행



PHASE 5 · 보조 분석 ⑧ Tuned Lens — Logit Lens 결과 보강용, 어느 단계에서나 병행 가능

Cell A vs Cell C 분석

핵심 질문

Polarity cue가 hate target 처리의 전제 조건인가?

연결 가설

H1 · 혐오 표현과 일반 부정문은 서로 다른 내부 활성화 양상을 보일 것이다

왜 지금 필요한가

현재 A vs B 비교는 trivial한 신호량 차이로도 설명 가능. 같은 hate target을 두고 cue 유무만 다른 A vs C 비교가 implicit hate 오분류 메커니즘에 직접 답한다. 또한 파일럿에서 행동 수준으로 확인된 'cue 없으면 positive로 오판단' 현상의 내부 표상 수준 검증.

Cell A vs Cell C 분석

절차

- 1 Cell A · Cell C minimal pair에 동일한 sentiment 프롬프트 접미
- 2 Layer 0-28 각각에서 Logit Difference 측정
- 3 쌍별 Cosine Distance와 5-fold Linear Probing 병행
- 4 A vs D 결과와 직접 비교 (drop ratio · selectivity 패턴)

측정 지표

`logit_diff(A) - logit_diff(C), cos_dist, probing acc + selectivity`

산출물: 레이어별 A-C gap 곡선 · A-D와 비교 plot

Cell A vs Cell C 분석

예상 결과

A vs C gap이 A vs B gap보다 작거나, 다른 레이어 분포로 나타날 것. 후반 레이어에서 두 셀이 유사한 표상으로 수렴할 가능성 (모델이 cue 없으면 hate target 무시).

결과 별 해석

- ① Gap이 양수로 유지 → 모델은 target만으로도 어느 정도 처리
- ② Gap이 0에 가까움 → 모델은 cue가 없으면 hate를 처리하지 못함 (오분류 메커니즘 입증)
- ③ Layer별 다른 분기점 → cue와 target이 다른 단계에서 처리됨

⚠ 예상 어려움 / 대응: Cell C 품질 (implicit hate가 너무 약하면 noise) → ToxiGen score 0.5 이상 샘플만 사용

Target Token 위치 분석

핵심 질문

Mean pooling이 묻어버린 target 처리 차이를 드러낸다

연결 가설

H1 · target-related content 효과를 위치 수준에서 분리

왜 지금 필요한가

현재 cosine distance는 문장 전체 평균을 보는데, 만약 target token 위치에서만 큰 차이가 있고 다른 토큰은 비슷하면 평균에서 묻혀버림. Tigges et al. (2023)의 'summarization motif' — sentiment가 특정 위치에 요약된다는 발견과 직접 연결. Target 토큰만 추출하면 평균에 묻힌 신호를 회복할 수 있음.

Target Token 위치 분석

절차

- 1 각 minimal pair에서 target token 위치 식별 (tokenizer offset 활용)
- 2 Mean pooling 대신 target 위치 hidden state만 추출
- 3 Layer 0-28 cosine distance · logit lens 재측정
- 4 마지막 토큰 위치 결과와 비교 (어디서 차이 발생하는지)

측정 지표

`cos_dist@target, logit_lens@target, norm@target`

산출물: 토큰 위치별 거리 heatmap · 전체 평균 vs target-only 곡선 비교

Target Token 위치 분석

예상 결과

Target token 위치에서 cosine distance가 mean pooling보다 큰 차이로 발생. 후반 레이어에서 특히 두드러질 것
— 모델이 target 정보를 그 위치에 누적하는 패턴.

결과 별 해석

- ① Target에서만 큰 차이, 다른 토큰 무관 → '국소적 hate 처리' 가설 지지
- ② Last token도 큰 차이 → 'summarization motif' 작동
- ③ 차이 없음 → 평균 분석이 적절했음 (오히려 좋은 sanity check)

⚠ 예상 어려움 / 대응: Target span이 multi-token (예: 'Black people') → 평균 또는 마지막 토큰 위치 사용

Sentiment Direction Projection

핵심 질문

표상은 비슷한데 예측은 다른 puzzle을 직접 설명

연결 가설

H3 · 유해성 신호와 단순 부정성 신호를 구분하는 보조 특징

왜 지금 필요한가

현재 결과의 핵심 puzzle: cosine distance는 작는데 logit gap은 큼. 가능한 답 — 두 표상이 전체적으로는 비슷하지만 sentiment를 담당하는 특정 1차원 방향에서만 차이가 큼. Tigges et al. (2023)이 sentiment의 선형 표상을 입증했으므로, 이 방향을 직접 추출해서 우리 셀들이 그 방향 위에서 어떻게 다른지 측정 가능.

Sentiment Direction Projection

절차

- 1 Sentiment 학습 데이터 (positive/negative reviews) 준비
- 2 각 레이어에서 $v_sent = \text{mean}(h+) - \text{mean}(h-)$ 추출
- 3 Cell A · C · D 각 샘플 hidden state를 v_sent 에 projection
- 4 Projection 값의 분포 비교 (셀 간 평균/분산 차이)

측정 지표

$$h \cdot v_sent / |v_sent|, \text{ layer-wise distribution}$$

산출물: 셀별 sentiment direction projection 분포 · 다른 방향들과 비교

Sentiment Direction Projection

예상 결과

Cell A가 sentiment direction 위에서 가장 negative, Cell D가 중간, Cell C가 positive 쪽으로 분포할 가능성. 후반 레이어에서 분리가 명확해질 것.

결과 별 해석

- ① 명확한 1차원 분리 → '표상은 비슷하지만 sentiment 축에서만 다름' 가설 입증
- ② 분리 약함 → sentiment 외 다른 방향에 인코딩됨 (다른 direction 탐색 필요)
- ③ Cell C가 Cell A 쪽으로 → 모델이 implicit hate도 어느 정도 잡고 있음

⚠ 예상 어려움 / 대응: Direction 추출용 데이터 부족 시 → 기존 sentiment 데이터셋 (SST, IMDB) 활용

Sentiment vs Harmfulness Direction 분리

핵심 질문

A-B gap이 어휘 효과인가, 별도 harmfulness 신호인가를 직접 분리

연결 가설

$H_{\text{contextual}} \cdot \text{sentiment}$ 방향과 harmfulness 방향이 layer 8 이후 분화한다

왜 지금 필요한가

Zhao et al. (2025)은 LLM이 harmfulness와 refusal을 별개 방향으로 인코딩함을 증명했다. 이를 우리 연구에 적용하면: Cell B에서 sentiment direction과 harmfulness direction을 각각 추출 후 두 방향의 cosine similarity를 레이어별로 측정한다. layer 8 이전에는 두 방향이 겹치다가 이후 분리된다면 — layer 8이 sentiment와 harmfulness를 분화시키는 지점이라는 mechanistic 해석이 성립한다. 이것이 A-B gap의 원인을 "단순 어휘 효과"에서 분리하는 직접적 방법이다.

Sentiment vs Harmfulness Direction 분리

절차

- 1 Cell B hidden state에서 sentiment direction 추출: $v_sent = \text{mean}(h_+) - \text{mean}(h_-)$
- 2 harmfulness direction 추출: harmful/harmless 쌍으로 $v_harm = \text{mean}(h_harm) - \text{mean}(h_safe)$
- 3 레이어별 $\text{cosine_sim}(v_sent, v_harm)$ 계산 (0~28 전체)
- 4 두 방향의 분화 시점 특징 — layer 8 이전/이후 비교

측정 지표

$\text{cosine_sim}(v_sent, v_harm)$ per layer · 분화 시점 (layer k^*)

산출물: 레이어별 방향 유사도 곡선 · 분화 시점 layer k^* · $H_lexical$ vs $H_contextual$ 판정

Sentiment vs Harmfulness Direction 분리

예상 결과

layer 0-7 구간에서 v_{sent} 와 v_{harm} 의 cosine sim이 높게 유지되다가 layer 8 이후 급격히 감소. 이는 초기에 두 방향이 어휘 수준에서 혼재하다가 layer 8에서 분화함을 의미한다.

결과 별 해석

- ① layer 8 이후 분화 → $H_{\text{contextual}}$ 지지. A-B gap이 harmfulness 신호에서 기인함
- ② 전 레이어 분화 없음 → H_{lexical} 지지. slur 어휘 효과가 지배적
- ③ layer 20 이후 재수렴 → 후반 레이어에서 두 신호가 통합되는 새 가설

⚠ 예상 어려움 / 대응: harmfulness 쌍 구성 어려울 시 → HateXplain hate/normal 쌍 활용. 참고: Zhao et al. (2025) NeurIPS

Cue 토큰 위치 Sentiment 표상 분석

핵심 질문

slur 어휘 효과와 contextual modulation을 토큰 위치 수준에서 분리

연결 가설

H_contextual · hate target이 cue 토큰 위치의 sentiment 표상에 영향을 준다

왜 지금 필요한가

Tigges et al. (2024)의 summarization motif: sentiment는 감정 단어뿐 아니라 심표·이름 등 중립 위치에도 요약된다. 이를 적용하면: H_lexical이라면 slur 토큰 위치에서만 A-B 차이가 발생해야 한다. H_contextual이라면 slur와 무관한 polarity cue 토큰 위치에서도 차이가 나타나야 한다. 두 위치를 비교하면 어느 가설이 맞는지 직접 판별 가능하다. 이것이 Cell C 없이 contextual modulation을 검증하는 핵심 방법이다.

Cue 토큰 위치 Sentiment 표상 분석

절차

- 1 v_{sent} 추출 (실험 ③-A와 동일 방법)
- 2 Cell A · Cell B 각 샘플에서 토큰 위치 식별: ① polarity cue 위치 ② slur(hate target) 위치
- 3 각 위치별 hidden state를 v_{sent} 에 projection: $\text{proj_cue} = h_{\text{cue}} \cdot v_{\text{sent}} / |v_{\text{sent}}|$
- 4 A-B 차이를 위치별로 비교: $\Delta\text{proj_cue}$ vs $\Delta\text{proj_slur}$ (레이어별)

측정 지표

$\Delta\text{proj_cue}(A-B)$, $\Delta\text{proj_slur}(A-B)$, layer-wise

산출물: 위치별 projection 차이 곡선 · cue vs slur 위치 비교 heatmap

Cue 토큰 위치 Sentiment 표상 분석

예상 결과

H_contextual 지지 시: cue 위치에서도 유의미한 Δproj 발생. 즉 hate target이 cue 토큰의 sentiment 표상에 영향을 미침. H_lexical 지지 시: slur 위치에서만 차이 발생, cue 위치는 A-B 동일.

결과 별 해석

- ① cue + slur 모두 차이 → contextual modulation 확인. A-B gap이 전체 문맥 처리에서 기인
- ② slur만 차이, cue 무관 → H_lexical 지지. 어휘 효과가 지배적 (실험 ③-A와 교차 검증)
- ③ cue 위치 차이가 후반 레이어에서만 발생 → summarization motif: 중간 레이어에서 통합

⚠ **예상 어려움 / 대응:** 토큰 위치 식별 어려울 시 → tokenizer offset 활용. multi-token span은 마지막 토큰 기준. 참고: Tigges et al. (2024) BlackboxNLP

Representation Engineering — Steering 수렴 검증

핵심 질문

Sentiment direction으로 A를 steering하면 B처럼 변하는가 — 인과적 분리

연결 가설

H_lexical vs H_contextual의 인과적 판별 — steering으로 직접 검증

왜 지금 필요한가

Representation Engineering (Zou et al., 2023)의 핵심 아이디어: 개념이 선형 방향으로 인코딩된다면, 그 방향 벡터를 더하거나 빼서 모델 행동을 조작할 수 있다. 이를 적용하면: Cell B의 hidden state에서 sentiment direction(v_{sent})을 추출 후 Cell A의 activation에 $-v_{\text{sent}}$ 를 추가한다. H_lexical이라면 steering으로 A의 logit diff가 B 수준으로 수렴해야 한다. H_contextual이라면 sentiment direction만으로는 수렴하지 않고 harmfulness direction 제거가 추가로 필요하다. 이것이 세 실험 중 가장 인과에 가까운 검증이다.

Representation Engineering — Steering 수렴 검증

절차

- 1 v_{sent} 추출 (실험 ③-A와 동일). Cell B 기준으로 추출
- 2 Steering: Cell A forward pass 중 layer k에서 $h_k \rightarrow h_k - \alpha \cdot v_{\text{sent}}$ 적용
- 3 Steered logit diff 측정: $\text{LD}_{\text{steered}}(\text{A})$ vs $\text{LD}(\text{B})$ 비교
- 4 수렴 실패 시 v_{harm} 추가 제거: $h_k - \alpha \cdot v_{\text{sent}} - \beta \cdot v_{\text{harm}} \rightarrow$ 재측정

측정 지표

$$\text{LD}_{\text{steered}}(\text{A}), \text{수렴률} = |\text{LD}_{\text{steered}} - \text{LD}(\text{B})| / |\text{LD}(\text{A}) - \text{LD}(\text{B})|$$

산출물: steering 전후 logit diff 분포 · v_{sent} 만 vs $v_{\text{sent}} + v_{\text{harm}}$ 비교

Representation Engineering — Steering 수렴 검증

예상 결과

v_{sent} steering만으로는 A가 B에 부분 수렴 (수렴률 40-60%). $v_{\text{sent}} + v_{\text{harm}}$ 함께 제거 시 수렴률이 80% 이상으로 상승. 이는 A-B gap이 sentiment + harmfulness 두 방향에 분산되어 있음을 시사한다.

결과 별 해석

- ① v_{sent} 만으로 완전 수렴 (>80%) $\rightarrow H_{\text{lexical}}$. 어휘 sentiment 효과가 지배적
- ② v_{sent} 부분 수렴, v_{harm} 추가 시 완전 수렴 $\rightarrow H_{\text{contextual}}$. 두 신호가 분리 인코딩됨
- ③ 어떤 steering도 수렴 실패 $\rightarrow$ 고차원 비선형 표상. linear hypothesis 한계

⚠ **예상 어려움 / 대응:** α 강도 조절 어려울 시 $\rightarrow$ 그리드 서치 ($\alpha \in \{5, 10, 20, 50\}$). off-distribution 방지 위해 perplexity 모니터링. 참고: Zou et al. (2023) RepE · Tigges et al. (2024)

Activation Patching

핵심 질문

Layer 8 분기점의 인과성을 검증한다

연결 가설

H2 · 특정 레이어에서 cue-target 정보 통합이 일어난다 (인과 검증)

왜 지금 필요한가

현재 Logit Lens는 'layer 8 이후 차이가 형성된다'를 보여주지만 이건 상관관계 수준. '바로 그 레이어가 차이를 만든다'를 입증하려면 직접 개입이 필요. Patching으로 layer 8의 activation을 다른 셀 것으로 교체했을 때 예측이 따라가는지 확인하면 인과 입증.

Activation Patching

절차

- 1 Clean run (Cell A) · Corrupted run (Cell C) 각각 forward pass 기록
- 2 Patched run: Cell A 입력 중 layer 8 hidden state를 Cell C로 교체
- 3 $\text{Effect} = (\text{logit_patched} - \text{logit_clean}) / (\text{logit_corrupted} - \text{logit_clean})$
- 4 Position 단위로도 patching (last token · target token 별)

측정 지표

patching_effect, ΔLD , recovery_ratio

산출물: Layer 8 patching effect 분포 · target vs other position 비교

Activation Patching

예상 결과

Layer 8 patching 시 effect가 크게 나타날 것 (0.5 이상). Effect가 작으면 layer 8은 표면적 상관만 있는 것. Position별로는 target token에서 가장 큰 effect 예상.

결과 별 해석

- ① Effect 큼 (>0.5) → Layer 8이 인과적 분기점, H2 지지
- ② Effect 작음 → 다른 레이어가 진짜 분기점 (sweep 필요)
- ③ Target position에서만 큼 → 'cue-target 통합' 가설 직접 입증

⚠ 예상 어려움 / 대응: Patching 구현 복잡 → TransformerLens 또는 nnsight 라이브러리 활용

Layer-by-layer Patching Sweep

핵심 질문

모든 레이어를 **systematic**하게 검증한다

연결 가설

H2 · 분기점이 정확히 어느 레이어인지 인과 수준에서 확정

왜 지금 필요한가

Layer 8만 patching하면 'layer 8이 분기점인가'에만 답함. 모든 레이어(0~28)를 하나씩 patching하면 어느 레이어가 가장 큰 effect를 갖는지 분포를 직접 볼 수 있음. Meng et al. (2022) ROME에서 mid-layer MLP가 결정적이었던 패턴이 우리 task에서도 재현되는지 확인.

Layer-by-layer Patching Sweep

절차

- 1 Layer 0부터 28까지 각각 single-layer patching
- 2 각 레이어에서 effect 측정 (165 pairs 평균 + bootstrap CI)
- 3 Effect 곡선을 Logit Lens gap 곡선과 비교
- 4 MLP-only patching · Attention-only patching 분리 (component-level)

측정 지표

effect(k) for k in $[0, 28]$, CI bands, attn vs mlp split

산출물: 레이어별 effect 곡선 + component decomposition

Layer-by-layer Patching Sweep

예상 결과

Layer 6-10 구간에서 effect peak. Logit Lens gap 시작 시점과 일치하면 patching 결과가 lens 결과를 인과적으로 뒷받침. MLP가 attention보다 더 큰 기여 가능성 (ROME 결과와 정합).

결과 별 해석

- ① Layer 6-10에서 peak, lens와 일치 → 강한 인과 증거
- ② Peak이 다른 곳 → 우리 lens 결과 재해석 필요
- ③ 여러 layer 분산 → 단일 분기점 아닌 분산 처리

⚠ 예상 어려움 / 대응: $165 \text{ pairs} \times 29 \text{ layers} = 4,785 \text{ forward pass}$ → batch 처리로 시간 단축

Causal Tracing 2D Map

핵심 질문

레이어 × 토큰 위치 동시 특정

연결 가설

H2 · cue-target 정보 통합 단계의 정확한 위치를 mechanistic 수준에서 식별

왜 지금 필요한가

Layer만 패칭하면 '언제'만 알 수 있고 '어디서'는 모름. 2D causal tracing은 (layer, position) 모든 조합을 patching하여 정보 흐름의 시작점과 경로를 함께 식별. Meng et al. (2022)이 subject token의 mid-layer에서 effect가 집중되는 패턴을 보였는데, 우리 task에서는 hate target에서 같은 패턴이 나올 가능성.

Causal Tracing 2D Map

절차

- 1 토큰 위치 정의: cue token · target token · last token
- 2 각 (layer k, position t)에 single patching
- 3 $29 \times 3 = 87$ 조합에 대한 effect heatmap 생성
- 4 정보 흐름 경로 식별: target → last 흐름이 어느 layer에서 발생하나

측정 지표

effect(k, t), 2D heatmap, information flow path

산출물: Layer × Position effect heatmap · 정보 흐름 다이어그램

Causal Tracing 2D Map

예상 결과

Target token 위치의 mid-layer에서 effect 집중. 그 이후 last token 위치로 정보가 흐르는 패턴 (ROME에서 발견된 'subject-to-last' 흐름의 hate 버전).

결과 별 해석

- ① Target@layer 6-10 + Last@layer 12-20 패턴 → cue-target 통합 후 last token으로 라우팅 (가장 깔끔한 결과)
- ② Target에서만 집중 → 통합은 target에서 완료, last는 단순 readout
- ③ 분산 → 단일 회로 없음

⚠ 예상 어려움 / 대응: Position alignment (target token 위치 식별) → 데이터 전처리에 metadata 추가

Attention Knockout

핵심 질문

Hate target 처리를 담당하는 핵심 attention head 식별

연결 가설

H2 두 번째 절 · 특정 attention head에서 차이가 두드러진다

왜 지금 필요한가

Wang et al. (2022) IOI 회로처럼, hate 처리에도 특정 head subset이 기여할 가능성. Layer 8 근처의 head를 하나씩 마스킹 해서 logit difference 변화 측정 → 핵심 head 식별. Tigges et al. (2023)도 sentiment를 담당하는 head subset이 소수임을 보였으므로, 우리 task에서도 비슷한 sparsity 예상.

Attention Knockout

절차

- 1 Layer 6-12 범위의 모든 head 식별 (Llama-3.2-3B: 24 heads/layer)
- 2 각 (layer, head)에 대해 attention output을 0으로 마스킹
- 3 $\Delta LD(A-C)$ 측정: head 제거 시 logit gap이 얼마나 줄어드는가
- 4 Top-K head로 회로 구성 후 그들만 활성화한 minimal model 검증

측정 지표

ΔLD per (layer, head), head importance ranking, minimal circuit

산출물: Head importance heatmap · top-K head의 attention pattern 시각화

Attention Knockout

예상 결과

Layer 6-10의 3-5개 head가 hate 처리에 집중적으로 기여. 그 head들의 attention pattern은 last token이 hate target을 주목하는 형태 (target-attending heads).

결과 별 해석

- ① 소수 head sparsity (3-5개) → hate 처리 회로 식별, novel contribution
- ② 많은 head 분산 (>10) → 분산 표상, 회로보다 분포적 처리
- ③ 특정 한 head 지배 → IO처럼 깔끔한 회로 (가장 강력한 결과)

⚠ 예상 어려움 / 대응: Head 수 많아 연산 비용 → 우선 layer 6-12로 범위 제한, 결과 보고 확대

Tuned Lens

핵심 질문

Logit Lens 결과의 신뢰성을 강화하는 보조 분석

연결 가설

방법론적 보강 · H1/H2의 결론을 강화

왜 지금 필요한가

현재 Logit Lens 결과에 oscillation이 있고, Belrose et al. (2023)은 Llama 계열에서도 Logit Lens가 일부 레이어에서 부정확함을 지적함. Tuned Lens로 같은 결과가 재현되면 우리 결론이 더 robust. 단독 contribution이 아니라 다른 실험의 보조 도구로 활용.

Tuned Lens

절차

- 1 공개된 Llama-3.2-3B용 Tuned Lens 가중치 확인 또는 직접 학습
- 2 학습 시: 일반 텍스트 데이터로 각 레이어 affine 변환 fit
- 3 Tuned Lens로 모든 셀(A, B, C, D)에 대해 예측 궤적 재산출
- 4 Logit Lens 결과와 직접 비교 (어느 레이어에서 차이가 큰지)

측정 지표

`tuned_lens(h_k)`, layer-wise prediction, lens 간 일치도

산출물: Tuned vs Logit Lens 비교 plot · 일치하는 레이어 / 차이 나는 레이어

Tuned Lens

예상 결과

Logit Lens의 핵심 발견(layer 8 분기점)은 Tuned Lens에서도 재현. 단, 초기 레이어와 일부 noise oscillation은 Tuned Lens에서 더 깔끔해짐. 분기점이 약간 shift할 수도 있음 (예: layer 7 또는 9).

결과 별 해석

- ① Layer 8 분기점 그대로 재현 → 결론 robust 확인
- ② 분기점 shift (1-2 layer) → 더 정밀한 위치 추정
- ③ 완전히 다른 패턴 → Logit Lens 결과 재검토 필요

⚠ 예상 어려움 / 대응: Tuned Lens 학습 데이터 부담 → 공개 weights 우선 시도, 없으면 학습

예상 어려움과 대응책 (종합)

8개 실험을 진행하면서 마주칠 공통 이슈

데이터 품질

Cell C의 implicit hate 강도가 약함 (bimodal 분포)

→ ToxiGen score ≥ 0.5 필터링 + 수동 검수 50개 샘플링

연산 비용

Patching sweep · Causal tracing은 $N_{\text{layer}} \times N_{\text{pos}} \times N_{\text{samples}}$

→ Batch 처리 + 165 pairs 중 100개만 사용 · Phase 3 부터 점진적

라이브러리 의존

Activation patching · attention knockout 구현 복잡

→ TransformerLens · nnsight 라이브러리 활용 · 자체 구현 x

결과 해석

Negative result (gap 없음, peak 없음 등) 가능성

→ 각 실험마다 ①②③ 시나리오 미리 정의 · 모든 결과가 의미 있도록 설계

재현성

Llama-3.2-3B 특수성 → 다른 모델에선 다를 수 있음

→ 최종 결과는 Llama-3.2-3B/7B/Qwen 등 2-3 모델에서 재현 확인

■ REFERENCES

우리 연구가 직접 기반하는 5편

이 5편을 중심으로 두고 논문 framing

1

Tigges et al. (2023)

Linear Representations of Sentiment in LLMs

이론적 기반 — sentiment의 선형 표상

2

Meng et al. (2022)

Locating and Editing Factual Associations in GPT

Activation patching 표준 방법론

3

Hewitt & Liang (2019)

Designing and Interpreting Probes with Control Tasks

Selectivity 지표

4

Zhang et al. (2024)

Don't Go To Extremes

LLM implicit hate 한계 — 연구 동기

5

ElSherief et al. (2021)

Latent Hatred

Implicit hate 정의 · taxonomy

■ REFERENCES

방법론별 표준 참고문헌

각 분석 기법의 원조 또는 표준 논문

Logit Lens	<i>nostalgebraist (2020) · Belrose et al. (2023)</i>
Tuned Lens	<i>Belrose et al. (2023)</i>
Probing	<i>Tenney et al. (2019) · Alain & Bengio (2016)</i>
Selectivity	<i>Hewitt & Liang (2019)</i>
Activation Patching	<i>Meng et al. (2022) · Vig et al. (2020)</i>
Logit Difference	<i>Wang et al. (2022) — IOI circuit</i>
Attention 분석	<i>Clark et al. (2019) · Abnar & Zuidema (2020) · Jain & Wallace (2019)</i>
Linear Representation	<i>Tigges et al. (2023) · Park et al. (2024) · Marks & Tegmark (2023)</i>